

ΜΥΕ023 – ΠΑΡΑΛΛΗΛΑ ΣΥΣΤΗΜΑΤΑ ΚΑΙ
ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ 2023-24

GPU PROGRAMMING

ΣΕΤ ΑΣΚΗΣΕΩΝ #2

Περιεχόμενα

Εισαγωγή	2
Άσκηση 1: cuinfo.....	3
Το πρόβλημα	3
Ανάλυση Κώδικα	4
Άσκηση 2: Θόλωση εικόνων με GPU offloading	8
Το πρόβλημα	8
Ανάλυση Κώδικα omp_device.....	9
Γραφικές Παραστάσεις - Χρονομετρήσεις.....	11
Συμπεράσματα.....	17

Εισαγωγή

Η 2^η σειρά ασκήσεων εστιάζει στον προγραμματισμό GPU και αποτελείται από δύο μέρη, καθένα από τα οποία αποσκοπεί να εξετάσει διαφορετικές πτυχές αυτής της τεχνολογίας.

Το πρώτο μέρος, με τίτλο "cuinfo", απαιτεί την ανάπτυξη ενός προγράμματος που θα ανιχνεύει και θα παρουσιάζει βασικές πληροφορίες για τις συσκευές CUDA που είναι διαθέσιμες στο σύστημα, όπως η ονομασία των συσκευών, οι εκδόσεις του CUDA compute capability και runtime, καθώς και πληροφορίες για τη μνήμη και τους streaming multiprocessors.

Το δεύτερο μέρος, με τίτλο "Θόλωση εικόνων με GPU offloading", περιλαμβάνει την υλοποίηση ενός προγράμματος θόλωσης εικόνων χρησιμοποιώντας Gaussian blur με την εκμετάλλευση της υπολογιστικής δύναμης της GPU. Ο στόχος είναι να μεταφερθεί η εκτέλεση του Gaussian blur από την CPU στην GPU, χρησιμοποιώντας το OpenMP για την αξιοποίηση των πολλών πυρήνων της GPU.

Με τον τρόπο αυτό, η εργασία στοχεύει να προσφέρει μια πρακτική εισαγωγή στη χρήση των GPUs για βελτίωση της απόδοσης των υπολογιστικών διεργασιών μέσω παραλληλοποίησης, ενισχύοντας την κατανόηση της χρήσης του CUDA runtime API και της τεχνολογίας GPU offloading.

Όλες οι μετρήσεις έγιναν στο παρακάτω σύστημα:

Όνομα υπολογιστή	parallax
Cpu	Intel(R) Xeon(R) Gold 6130 CPU @ 2.10GHz
Πλήθος πυρήνων	64
Gpu	Device 0: Nvidia Tesla P40 (Architecture:Pascal)
Μεταφραστής	gcc (GCC) 11.2.1 20220302

Άσκηση 1: cuinfo

Το πρόβλημα

Το πρόβλημα που τίθεται στην άσκηση 1 αφορά την ανίχνευση και παρουσίαση των πληροφοριών των συσκευών CUDA που είναι διαθέσιμες σε ένα σύστημα, παρόμοια με την λειτουργικότητα του utility clinfo για συσκευές OpenCL.

Το ζητούμενο είναι η ανάπτυξη ενός προγράμματος με την ονομασία cuinfo, το οποίο θα χρησιμοποιεί το CUDA Runtime API για να ανακαλύπτει τις συσκευές CUDA και να εμφανίζει συγκεκριμένες πληροφορίες για κάθε μία από αυτές:

- όπως το όνομα της συσκευής
- την έκδοση CUDA compute capability
- την έκδοση του CUDA driver και runtime,
- τον αριθμό των διαθέσιμων streaming multiprocessors (SMs)
- το συνολικό μέγεθος της καθολικής μνήμης (global memory),
- το μέγεθος της σταθερής μνήμης (constant memory)
- το μέγεθος της κοινόχρηστης μνήμης (shared memory) ανά block.
- Επιπλέον, ζητείται να διερευνηθεί αν υπάρχει τρόπος να βρεθεί ο συνολικός αριθμός των cores μιας NVIDIA GPU και, αν ναι, να παρατεθούν οι σχετικές λεπτομέρειες στην αναφορά

Ανάλυση Κώδικα

- `cudaGetDeviceCount(&num_devs)`: Επιστρέφει τον αριθμό των διαθέσιμων συσκευών CUDA.
- `cudaGetDeviceProperties(&dev_prop, i)`: Επιστρέφει τις ιδιότητες της συσκευής με δείκτη `i`.
- Εκτύπωση των πληροφοριών: Οι πληροφορίες εκτυπώνονται με χρήση της `printf`.

Αυτός ο κώδικας θα εκτυπώσει τις ζητούμενες πληροφορίες για κάθε συσκευή CUDA που είναι εγκατεστημένη στο σύστημά σας.

```
// We retrieve and pretty-print all the necessary information
cudaGetDeviceProperties(&dev_prop, i);

// Primary information
printf("Primary information\n");
printf("Device %d: %s\n", i, dev_prop.name);
printf("CUDA Compute Capability: %d.%d\n", dev_prop.major, dev_prop.minor);
printf("CUDA Driver Version / Runtime Version: %d.%d / %d.%d\n",
       driver_version / 1000, (driver_version % 100) / 10,
       runtime_version / 1000, (runtime_version % 100) / 10);
printf("Number of SMs: %d\n", dev_prop.multiProcessorCount);
printf("Total Global Memory: %.2f GB\n", (float)dev_prop.totalGlobalMem / (1024 * 1024 * 1024));
printf("Total Constant Memory: %lu KB\n", dev_prop.totalConstMem / 1024);
printf("Shared Memory per Block: %lu KB\n", dev_prop.sharedMemPerBlock / 1024);
```

```
// Secondary information
printf("Secondary information\n");
printf("Warp Size: %d\n", dev_prop.warpSize);
printf("Maximum Threads per Block: %d\n", dev_prop.maxThreadsPerBlock);
printf("Maximum Threads per SM: %d\n", dev_prop.maxThreadsPerMultiProcessor);
printf("Maximum Grid Size: %d x %d x %d\n", dev_prop.maxGridSize[0], dev_prop.maxGridSize[1], dev_prop.maxGridSize[2]);
printf("Maximum Block Dimensions: %d x %d x %d\n", dev_prop.maxThreadsDim[0], dev_prop.maxThreadsDim[1], dev_prop.maxThreadsDim[2]);
```

```

// Calculate and print the total number of CUDA cores
// We extract information about the architecture based on the Compute Capability
int coresPerSM = 0;
switch (dev_prop.major) {
    case 1: // Tesla
        coresPerSM = 8;
        break;
    case 2: // Fermi
        coresPerSM = 32;
        break;
    case 3: // Kepler
        coresPerSM = 192;
        break;
    case 5: // Maxwell
        coresPerSM = 128;
        break;
    case 6: // Pascal
        coresPerSM = 128;
        break;
    case 7: // Volta or Turing
        coresPerSM = 64;
        break;
    case 8: // Ampere
        coresPerSM = 128;
        break;
    case 9: // Hopper
        coresPerSM = 128;
        break;
    default:
        printf("Unknown compute capability\n");
        continue;
}
int totalCores = coresPerSM * dev_prop.multiProcessorCount;
printf("Total CUDA Cores: %d\n", totalCores);

```

```
[ex24012@parallax ~]$ nvcc cuinfo.cu -o cuinfo
[ex24012@parallax ~]$ ./cuinfo
Primary information
Device 0: Tesla P40
CUDA Compute Capability: 6.1
  CUDA Driver Version / Runtime Version: 11.6 / 11.6
Number of SMs: 30
Total Global Memory: 22.38 GB
Total Constant Memory: 64 KB
Shared Memory per Block: 48 KB
Secondary information
Warp Size: 32
Maximum Threads per Block: 1024
Maximum Threads per SM: 2048
Maximum Grid Size: 2147483647 x 65535 x 65535
Maximum Block Dimensions: 1024 x 1024 x 64
Total CUDA Cores: 3840
```

Για το τελευταίο ζητούμενο, που αφορά την εύρεση του συνολικού αριθμού των cores μιας NVIDIA GPU, η απάντηση βασίζεται στον υπολογισμό που γίνεται με βάση τον αριθμό των streaming multiprocessors (SMs) και τον αριθμό των CUDA cores ανά SM για τη συγκεκριμένη αρχιτεκτονική της GPU.

Κάθε αρχιτεκτονική GPU της NVIDIA έχει έναν διαφορετικό αριθμό CUDA cores ανά SM.

Για να βρούμε τον συνολικό αριθμό των CUDA cores μιας NVIDIA GPU, μπορούμε να χρησιμοποιήσουμε τον εξής τύπο:

$\text{Total CUDA cores} = \text{Number of SMs} \times \text{CUDA cores per SM}$
--

Ο αριθμός των CUDA cores ανά SM εξαρτάται από την αρχιτεκτονική της GPU.

Ενδεικτικά παραδείγματα αρχιτεκτονικών:

- Tesla (Compute Capability 1.x):
Cores per SM: Ποικίλλει ανάλογα με τη συγκεκριμένη υποέκδοση, πιο σύνηθες 8.
- Fermi (Compute Capability 2.x):
Cores per SM: 32 (εκτός από μερικά πρώιμα μοντέλα που είχαν 48).
- Kepler (Compute Capability 3.x):
Cores per SM: 192.

- Maxwell (Compute Capability 5.x):
Cores per SM: 128.
- Pascal (Compute Capability 6.x):
Cores per SM: 128.
- Volta and Turing (Compute Capability 7.x):
64 CUDA cores per SM
- Ampere (Compute Capability 8.x):
Cores per SM: 128
- Hopper (Compute Capability 9.x):
Cores per SM: 128

Αν γνωρίζουμε την αρχιτεκτονική που χρησιμοποιείται μπορούμε να εξαγάγουμε απευθείας τον αριθμό των cores ανά SM .

Αν δεν διαθέτουμε αυτή την πληροφορία μπορούμε να εξαγάγουμε συμπεράσματα με βάση το Compute Capability όπως φαίνεται παραπάνω.

Άσκηση 2: Θόλωση εικόνων με GPU offloading

Το πρόβλημα

Το πρόβλημα που τίθεται στην άσκηση 2 είναι η μετατροπή ενός σειριακού πρόγραμματος θόλωσης εικόνας, που χρησιμοποιεί τη συνάρτηση `gaussian_blur_serial()`, σε ένα πρόγραμμα που αξιοποιεί τη GPU για την επιτάχυνση της επεξεργασίας μέσω OpenMP.

Η νέα συνάρτηση, `gaussian_blur_omp_device()`, πρέπει να χρησιμοποιεί τις κατάλληλες οδηγίες για offloading, ώστε η θόλωση να πραγματοποιείται στην GPU.

Θα πρέπει να πειραματιστούμε με διαφορετικές ρυθμίσεις για το πλήθος ομάδων και νημάτων ανά ομάδα, με τις εξής προϋποθέσεις:

το πλήθος νημάτων ανά ομάδα να είναι πολλαπλάσιο του 32,

και το πλήθος ομάδων να εξασφαλίζει τη χρήση όλων των πυρήνων της συσκευής.

Η συσκευή CUDA διαθέτει 30 streaming multiprocessors, με 128 πυρήνες ο καθένας.

Ανάλυση Κώδικα omp_device

Αρχικά με βάση τους περιορισμούς της άσκησης μπορούμε να ξεχωρίσουμε κατηγορίες πειραμάτων μαζί με τους συνδυασμούς τους.

Πρώτον καθώς θέλουμε το πλήθος των νημάτων ανά ομάδα να είναι πάντα πολλαπλάσιο του 32 και το πλήθος ομάδων να πρέπει να είναι τέτοιο ώστε όλοι οι πυρήνες της συσκευής (3840) να χρησιμοποιούνται πάντα μπορούμε να ξεχωρίσουμε 2 περιορισμούς .

1. Το ελάχιστο πλήθος ομάδων πρέπει να συμπίπτει με το πλήθος των SMs άρα 30.
2. Για να χρησιμοποιούνται πάντα και οι 3840 cores θα πρέπει το γινόμενο πλήθος ομάδων $\times$ νήματα ανά ομάδα να είναι πάντα μεγαλύτερο του 3840.

Δεύτερη κατηγορία, μπορούμε να ξεχωρίσουμε τη σύνταξη των εντολών σε 2 περιπτώσεις : εμφωλευμένες οδηγίες ή συνδυασμένες.

Με βάση τους περιορισμούς τα πειράματα που θα εξετάσουμε είναι:

Εμφωλευμένες οδηγίες / Συνδυασμένες οδηγίες
30 ομάδες / 128, 256, 512, 1024 νήματα ανά ομάδα
60 ομάδες / 64 , 128, 256, 512 νήματα ανά ομάδα
120 ομάδες/ 32, 64, 256, 256 νήματα ανά ομάδα

Παρακάτω παραθέτω δύο παραδείγματα κώδικα με διαφορά ως προς την σύνταξη των οδηγιών:

Εμφωλευμένες οδηγίες:

```
int img_size = width*height; // new

#pragma omp target map(to: imgin->red[0:img_size], imgin->green[0:img_size], imgin->blue[0:img_size], radius, width, height)\
    map(from: imgout->red[0:img_size], imgout->green[0:img_size], imgout->blue[0:img_size])
{
    #pragma omp teams num_teams(30) thread_limit(1024)
    {
        #pragma omp distribute parallel for collapse(2)\
            private(i, j, col, row)\
            firstprivate(redSum, greenSum, blueSum, weightSum)
        for (i = 0; i < height; i++)
        {
            for (j = 0; j < width ; j++)
            {
```

Συνδυασμένες οδηγίες:

```
int img_size = width*height; // new

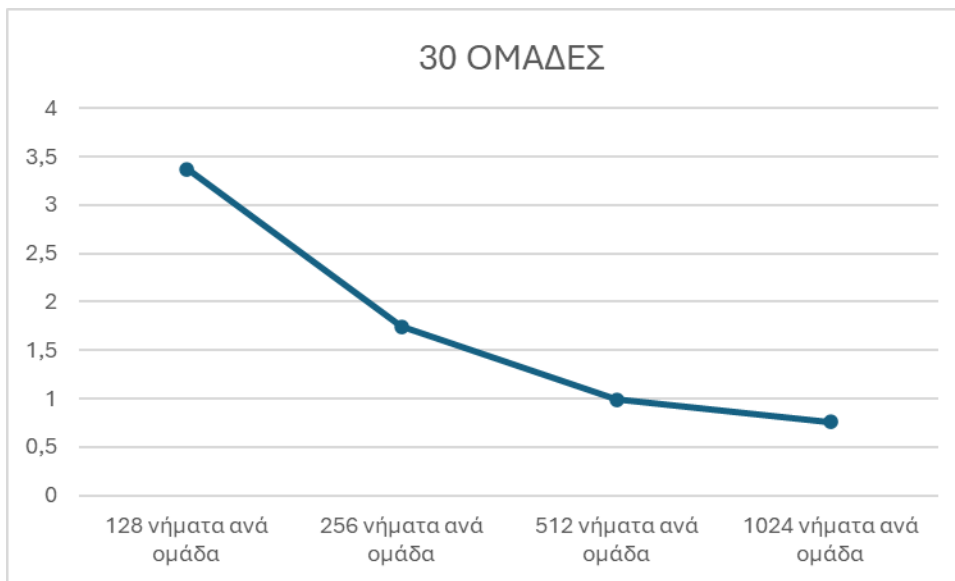
#pragma omp target teams distribute parallel for collapse(2) num_teams(120) num_threads(32)\
    map(to: imgin->red[0:img_size], imgin->green[0:img_size], imgin->blue[0:img_size], radius, width, height)\
    map(from: imgout->red[0:img_size], imgout->green[0:img_size], imgout->blue[0:img_size])\
    private(i, j, col, row)\
    firstprivate(redSum, greenSum, blueSum, weightSum)
for (i = 0; i < height; i++)
{
    for (j = 0; j < width ; j++)
    {
```

Παρατήρηση η χρήση της οδηγίας **num_threads** στις **εμφωλευμένες** οδηγίες **δεν λειτουργεί** όπως περιμέναμε (**ότι και να βάζαμε στο num_threads δημιουργούσε 128 νήματα ανά ομάδα**) , για αυτό θα χρησιμοποιήσουμε την οδηγία **thread_limit**.

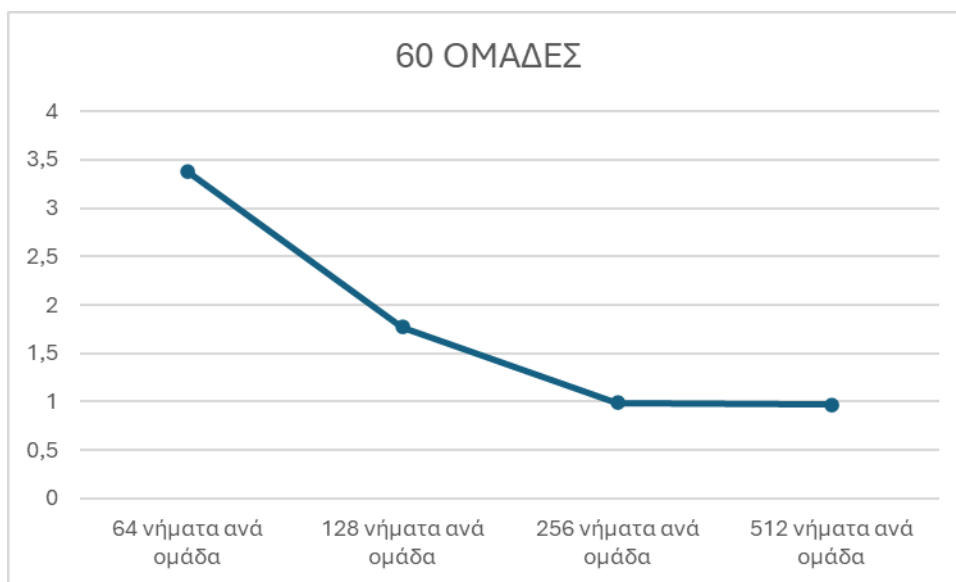
Γραφικές Παραστάσεις - Χρονομετρήσεις

- Σύνταξη οδηγιών εμφωλευμένα:

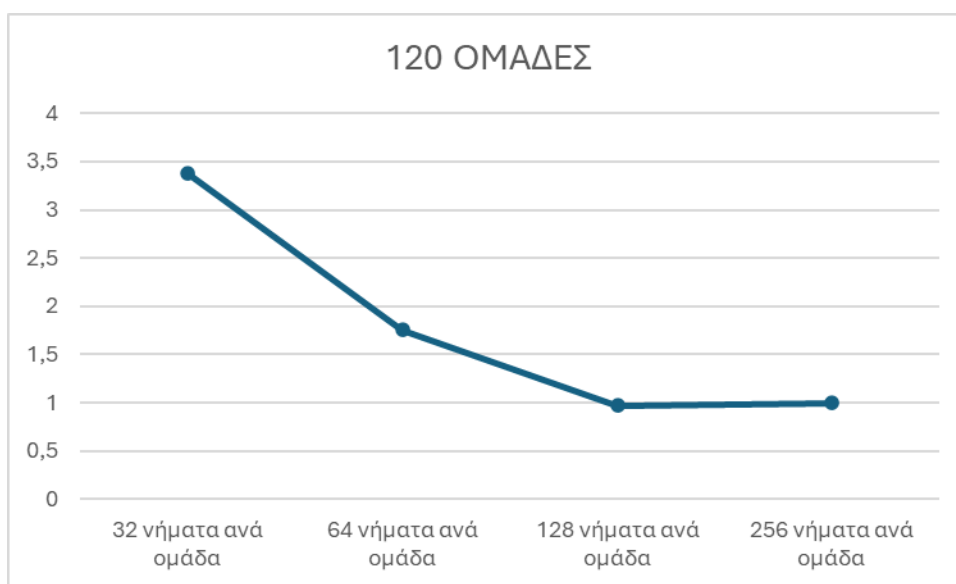
30 ομάδες	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	Μ.Ο
128 νήματα ανά ομάδα	3,38	3,36	3,39	3,37	3,375
256 νήματα ανά ομάδα	1,75	1,754	1,765	1,712	1,745
512 νήματα ανά ομάδα	0,987	0,986	0,983	1,00	0,989
1024 νήματα ανά ομάδα	0,7576	0,72	0,773	0,777	0,756



60 ομάδες	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	Μ.Ο
64 νήματα ανά ομάδα	3,365	3,379	3,387	3,384	3,378
128 νήματα ανά ομάδα	1,771	1,770	1,776	1,778	1,773
256 νήματα ανά ομάδα	0,965	1,00	0,993	0,992	0,987
512 νήματα ανά ομάδα	0,986	0,98	0,925	0,981	0,968

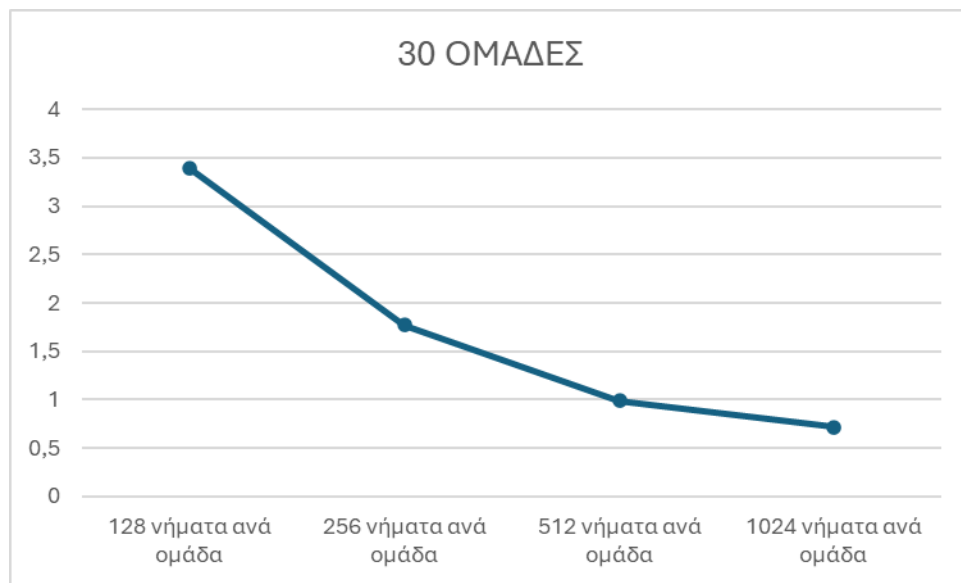


120 ομάδες	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	Μ.Ο
32 νήματα ανά ομάδα	3,39	3,388	3,327	3,389	3,373
64 νήματα ανά ομάδα	1,758	1,713	1,78	1,78	1,757
128 νήματα ανά ομάδα	0,978	0,971	0,977	0,959	0,971
256 νήματα ανά ομάδα	0,988	0,978	0,994	1,02	0,995

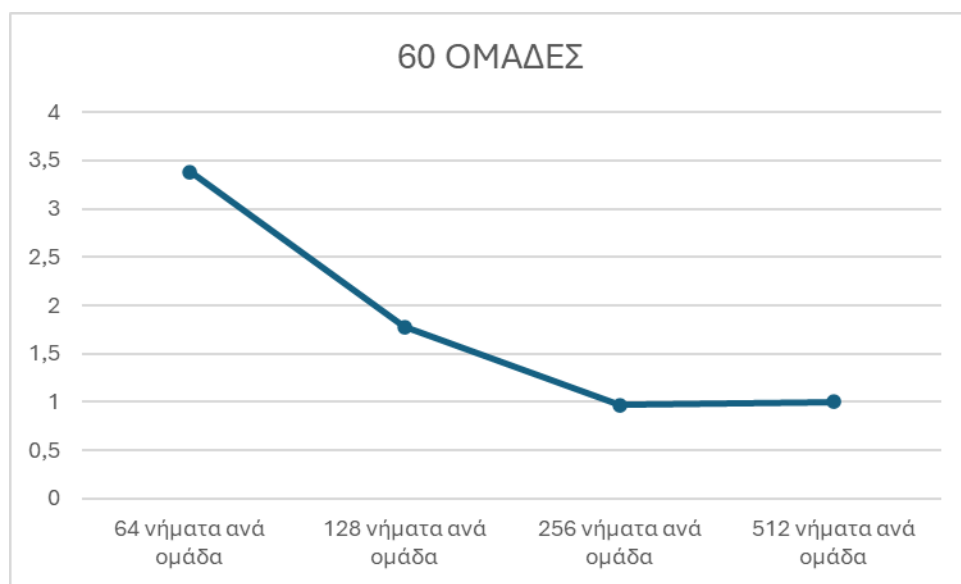


- Σύνταξη οδηγιών συνδιαστικά :

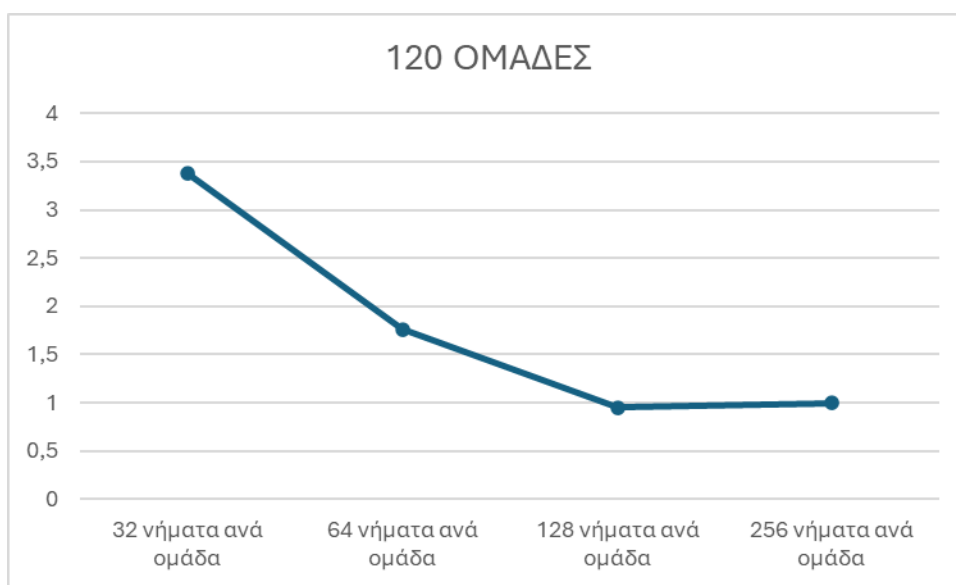
30 ομάδες	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	Μ.Ο
128 νήματα ανά ομάδα	3,387	3,381	3,389	3,398	3,388
256 νήματα ανά ομάδα	1,784	1,763	1,765	1,772	1,771
512 νήματα ανά ομάδα	0,985	0,981	0,999	0,980	0,986
1024 νήματα ανά ομάδα	0,72	0,715	0,72	0,72	0,718



60 ομάδες	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	Μ.Ο
64 νήματα ανά ομάδα	3,377	3,39	3,391	3,375	3,383
128 νήματα ανά ομάδα	1,787	1,778	1,784	1,773	1,78
256 νήματα ανά ομάδα	0,931	0,985	0,995	0,972	0,97
512 νήματα ανά ομάδα	0,95	0,93	1,22	0,93	1



120 ομάδες	1 ^η εκτέλεση	2 ^η εκτέλεση	3 ^η εκτέλεση	4 ^η εκτέλεση	Μ.Ο
32 νήματα ανά ομάδα	3,374	3,38	3,39	3,374	3,379
64 νήματα ανά ομάδα	1,777	1,764	1,778	1,719	1,759
128 νήματα ανά ομάδα	1,01	0,943	0,939	0,925	0,954
256 νήματα ανά ομάδα	1,04	0,98	1,004	0,995	1



Συμπεράσματα

1. Διαφορές - Ομοιότητες Εμφωλευμένης-Συνδυαστικής:

Πέρα από τον τρόπο που πρέπει να ορίσουμε το πλήθος νημάτων ανά ομάδα (num_threads και threads_limit) δεν παρατηρείται κάποια αξιοσημείωτη διαφορά ως προς την απόδοση.

2. Βέλτιστη Διαμόρφωση Νημάτων:

Η αύξηση του αριθμού των νημάτων ανά ομάδα γενικά οδηγεί σε καλύτερη απόδοση (μικρότερο μέσο χρόνο εκτέλεσης) σε όλες τις διαμορφώσεις ομάδων. Αυτό πιθανότατα οφείλεται στην καλύτερη αξιοποίηση των πυρήνων της GPU και στη μείωση του κόστους ανά νήμα.

Ωστόσο πρέπει να τονίσουμε ότι τα βέλτιστα αποτελέσματα δεν προκύπτουν από τις τιμές μίας και μόνο μεταβλητής αλλά από τον συνδυασμό του αριθμού των ομάδων με το πλήθος νημάτων ανά ομάδα.

Ενδεικτικά βέλτιστοι συνδυασμοί:

30 ομάδες / 512 και 1024 νήματα ανά ομάδα.

60 ομάδες / 256 και 512 νήματα ανά ομάδα.

120 ομάδες / 128 και 256 νήματα ανά ομάδα.

Ενδεικτική εκτέλεση (num_teams(30) threads_limit(1024)):

```
[ex24012@parallax ~]$ ./a.out 8 1500.bmp
<<< Gaussian Blur (h=1500,w=1500,r=8) >>>
Total execution time (sequential): 18.146096
Total execution time (omp device): 0.766844
```

3. Σταθερότητα στην Απόδοση:

Σε διαφορετικές διαμορφώσεις, η βελτίωση στον χρόνο εκτέλεσης τείνει να σταθεροποιηθεί σε υψηλότερους αριθμούς νημάτων ανά ομάδα.

Αυτό υποδηλώνει ότι υπάρχει ένα σημείο κορεσμού πέρα από το οποίο η προσθήκη περισσότερων νημάτων δεν βελτιώνει σημαντικά την απόδοση λόγω περιορισμών υλικού ή άλλων επιβαρύνσεων.

Σημείωση :

Σχόλια για τις εντολές που χρησιμοποιήσαμε έχουν προστεθεί πάνω στον παραδοτέο κώδικα .

Ο πλήθος των ομάδων και το πλήθος των νημάτων ανά ομάδα έχουν οριστεί σε μία από τις βέλτιστες περιπτώσεις.

Κρατήσαμε την εμφωλευμένη υλοποίηση χωρίς κάποιον ιδιαίτερο λόγο.